



Կառուցվածքային տիպերը C++ լեզվում

Գրառումներից մինչև զանգված, ցուցիչ և ֆունկցիա

Առարկա՝ ՕԿԾ, Դասախոս՝ Սոնյա Սահակյան



ԻՆՋՈՒԷ անհրաժեշտ կառուցվածքը

```
string name1 = "Ani";  
int age1 = 18;  
double grade1 = 16.5;  
  
string name2 = "Aram";  
int age2 = 19;  
double grade2 = 15.0;
```

Խնդիրը

Յուրաքանչյուր ուսանողի տվյալները պահվում են առանձին փոփոխականներում: Գրառումների քանակի աճով դժվարանում են պահպանումը և մշակումը:

Լուծման գաղափարը

Մեկ ուսանողին պատկանող տարբեր տիպերի տվյալները միավորում ենք մեկ Student գրառման մեջ:

Կառուցվածքային տիպի գաղափարը

Կառուցվածքը օգտագործողի սահմանած տիպ է, որը մեկ անվան տակ միավորում է տարբեր տիպերի փոխկապակցված տվյալներ:

Սահմանումը նկարագրում է գրառման կազմը:
Հիշողությունն հատկացվում է կառուցվածքային փոփոխական ստեղծելիս:

Student

name : string

age : int

grade : double

Մեկ ամբողջական գրառում



Կառուցվածքի հայտարարումը

```
struct TypeName {  
    dataType member1;  
    dataType member2;  
};
```

```
struct Student {  
    string name;  
    int age;  
    double grade;  
};
```

Կարևոր

Փակող փակագծից հետո պարտադիր դրվում է կետ ստորակետ:

Կառուցվածքային տիպի ստեղծումը

```
Student student1;
```

Հիշողության մեջ ստեղծվում է մեկ
Student գրառում

student1

name

age

grade

Յուրաքանչյուր Student տիպի օբյեկտի
ունի իր սեփական անդամները

Անդամների սկզբնարժեքավորումը

Անդամների արժեքների վերագրում

```
Student student1;  
student1.name = "Ani";  
student1.age = 18;  
student1.grade = 16.5;
```

Սկզբնարժեքավորում հայտարարման պահին

```
Student student2 = {  
    "Aram", 19, 15.0  
};
```

Արժեքների հերթականությունը պետք է համապատասխանի անդամների հայտարարման հերթականությանը

Անդամին դիմելը կետային գործողությամբ

```
student1.name  
student1.age  
student1.grade
```

student1 . **grade**

Ընթերցում

```
cout << student1.grade;
```

Փոփոխում

```
student1.grade = 18;
```

Կետը ընտրում է տվյալ գրառման
ներսում գտնվող անդամը

Մուտքագրում և արտածում

```
cout << "Name: ";  
cin >> student1.name;  
  
cout << "Age: ";  
cin >> student1.age;  
  
cout << "Grade: ";  
cin >> student1.grade;
```

```
cout << student1.name << " "  
      << student1.age << " "  
      << student1.grade;
```

Հիշել

Յուրաքանչյուր անգամ մուտքագրվում և արտածվում է առանձին: Բացառով անունների համար կիրառվում է getline:

Կառուցվածքների զանգվածը

```
Student students[3];
```

Յուրաքանչյուր տարր մեկ
ամբողջական Student գրառում է

students[0]

name
age
grade

students[1]

name
age
grade

students[2]

name
age
grade

students[2].grade երրորդ ուսանողի գնահատականը

Չանգվածի տվյալների մուտքագրումը

```
for (int i = 0; i < 3; i++) {  
    cout << "Name: ";  
    cin >> students[i].name;  
  
    cout << "Age: ";  
    cin >> students[i].age;  
  
    cout << "Grade: ";  
    cin >> students[i].grade;  
}
```

i = 0

Լրացվում է առաջին ուսանողի գրառումը:

i = 1

Լրացվում է երկրորդ ուսանողի գրառումը:

i = 2

Լրացվում է երրորդ ուսանողի գրառումը:

Չանգվածի տվյալների արտածումը

```
for (int i = 0; i < 3; i++) {  
    cout << students[i].name << " "  
        << students[i].age << " "  
        << students[i].grade << endl;  
}
```

Քայլերը

1. Ընտրել զանգվածի տարրը
2. Կետով ընտրել անհրաժեշտ անդամը
3. Կատարել արտածումը

Մեկ ցիկլը հերթով մշակում է զանգվածի բոլոր գրառումները

Ամենաբարձր գնահատականի որոնումը

```
int bestIndex = 0;

for (int i = 1; i < 3; i++) {
    if (students[i].grade >
        students[bestIndex].grade) {
        bestIndex = i;
    }
}

cout << students[bestIndex].name;
```

Քայլային ստուգում

Սկիզբ

bestIndex = 0
լավագույնը՝ Ani

i = 1

15.0 < 16.5
ինդեքսը մնում է 0

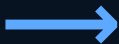
i = 2

17.5 > 16.5
ինդեքսը դառնում է 2

Կառուցվածքի ցուցիչ

```
Student student1 = {"Ani", 18, 16.5};  
Student* ptr = &student1;
```

ptr



student1

name = Ani
age = 18
grade = 16.5

Ցուցիչը պահում է գրառման հասցեն և մատնանշում նույն տվյալները

Կետ և սլաք գործողությունները

Սովորական փոփոխական

```
student1.name  
student1.grade
```

Կառուցվածքի ցուցիչ

```
ptr->name  
ptr->grade
```

```
ptr->grade
```

նույնն է, ինչ

```
(*ptr).grade
```

Ցուցիչ կիրառելիս պետք է համոզվել, որ այն ունի վավեր հասցե

Կառուցվածքի փոխանցումն արժեքով

```
void changeGrade(Student student) {  
    student.grade = 20;  
}
```

Ստեղծվում է պատճեն

Ֆունկցիան փոխում է տեղական
պատճենը: Սկզբնական
student1-ը մնում է անփոփոխ:

student1
grade = 16.5

→ պատճենում

պատճեն
grade = 20

Կառուցվածքի փոխանցումը հղումով

```
void changeGrade(Student& student) {  
    student.grade = 20;  
}  
  
changeGrade(student1);
```

Պատճեն չի ստեղծվում

student պարամետրը սկզբնական student1 գրառման այլ անունն է: Փոփոխությունը պահպանվում է ֆունկցիայի ավարտից հետո:

Ֆունկցիայից հետո

```
student1.grade == 20
```

Կառուցվածքի փոխանցումը ցուցիչով

```
void changeGrade(Student* student) {  
    if (student != nullptr) {  
        student->grade = 20;  
    }  
}  
  
changeGrade(&student1);
```

Կանչի ժամանակ

Փոխանցում ենք հասցեն՝
&student1:

Ֆունկցիայի ներսում

Անդամին դիմում ենք ->
գործողությամբ:

Անվտանգություն

nullptr-ը կանխում է անվավեր
հասցեի օգտագործումը:

Փոխանցման եղանակների համեմատությունը

Student s

ստեղծում է պատճեն

սկզբնականը չի փոխվում

Student& s

պատճեն չի ստեղծում

կարող է փոփոխել

const Student& s

պատճեն չի ստեղծում

միայն կարդում է

Student* s

ստանում է հասցեն

պահանջում է հասցեի
ստուգում

Տվյալները միայն արտաճելու համար կիրառեք **const Student&**



Ամփոփում

- Կառուցվածքը մեկ անվան տակ միավորում է փոխկապակցված տարբեր տիպերի տվյալներ:
- Կառուցվածքների զանգվածի յուրաքանչյուր տարր մեկ ամբողջական գրառում է:
- Սովորական գրառման անդամին դիմում ենք կետով, իսկ ցուցիչի դեպքում՝ սլաքով:
- Փոխանցման եղանակը ընտրում ենք ըստ նպատակի՝ պատճենել, փոփոխել կամ միայն կարդալ:

Առաջադրանք

Սահմանել Book կառուցվածք՝ title, author, price անդամներով: Ստեղծել 3 գրքի զանգված և գտնել ամենաթանկ գիրքը: